decision_tree.R

dhanushh

2026-01-07

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(rpart)
library(ggplot2)
head(diamonds)
```

```
## # A tibble: 6 x 10
##   carat cut       color clarity depth table price     x     y     z
##   <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
## 2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
## 3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
## 4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
## 5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
## 6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
```

```
nrow(diamonds) #no of entries
```

```
## [1] 53940
```

```
set.seed(94)
train_control = trainControl(method = "cv", number = 10)

diamonds$cut <- as.factor(diamonds$cut) #coerce target var as factor for cm

tree1 <- train(cut ~., data = diamonds, method = "rpart1SE", trControl = train_control)
tree1
```

```
## CART
##
## 53940 samples
##     9 predictor
##     5 classes: 'Fair', 'Good', 'Very Good', 'Premium', 'Ideal'
##
## No pre-processing
```

```
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 48545, 48545, 48546, 48545, 48547, 48546, ...
## Resampling results:
##
##   Accuracy   Kappa
##   0.7086764  0.5812043
```

```
#evaluate fit with a cm
pred_tree <- predict(tree1, diamonds)
confusionMatrix(diamonds$cut, pred_tree)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   Fair  Good Very Good Premium Ideal
##   Fair       1211   240        13     119    27
##   Good        161  2422      1053    1121   149
##   Very Good    16   746      2214    5498  3608
##   Premium       0     0         0   12099  1692
##   Ideal        14    24        57    1184 20272
##
## Overall Statistics
##
##                Accuracy : 0.7085
##                  95% CI : (0.7047, 0.7124)
##     No Information Rate : 0.4773
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.58
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Fair Class: Good Class: Very Good Class: Premium
## Sensitivity              0.86377     0.70571          0.66347         0.6043
## Specificity              0.99241     0.95082          0.80499         0.9501
## Pos Pred Value           0.75217     0.49368          0.18325         0.8773
## Neg Pred Value           0.99635     0.97940          0.97317         0.8027
## Prevalence               0.02599     0.06363          0.06187         0.3712
## Detection Rate           0.02245     0.04490          0.04105         0.2243
## Detection Prevalence     0.02985     0.09095          0.22399         0.2557
## Balanced Accuracy        0.92809     0.82827          0.73423         0.7772
##                      Class: Ideal
## Sensitivity                0.7873
## Specificity                0.9546
## Pos Pred Value             0.9407
## Neg Pred Value             0.8309
## Prevalence                 0.4773
## Detection Rate             0.3758
## Detection Prevalence       0.3995
## Balanced Accuracy          0.8710
```

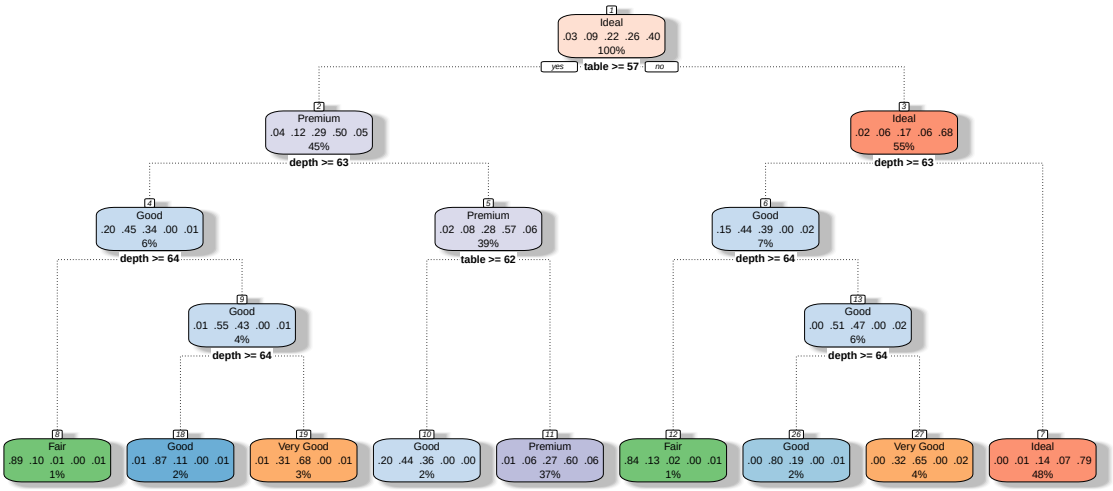
```
library(rattle) #install packages rattle and rpart.plot
```

```
## Loading required package: tibble
```

```
## Loading required package: bitops
```

```
## Rattle: A free graphical interface for data science with R.
## Version 5.5.1 Copyright (c) 2006-2021 Togaware Pty Ltd.
## Type 'rattle()' to shake, rattle, and roll your data.
```

```
#visualizing my decision tree
fancyRpartPlot(tree1$finalModel, caption = "")
```




```
train_control = trainControl(method = "cv", number = 10)
#fit the model
tree_caret <- train(cut ~., data = diamonds, method = "rpart", trControl = train_control)
tree_caret
```

```
## CART
##
## 53940 samples
##     9 predictor
##     5 classes: 'Fair', 'Good', 'Very Good', 'Premium', 'Ideal'
```

```
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 48545, 48547, 48546, 48546, 48546, 48546, ...
## Resampling results across tuning parameters:
##
##   cp          Accuracy   Kappa
##   0.04337892  0.6430857  0.4819975
##   0.05193121  0.6134212  0.4258976
##   0.33622526  0.4995736  0.1989137
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.04337892.
```

```
tree2 <- train(cut ~., data = diamonds, control = rpart.control(maxdepth = 3), trControl = train_control
tree2
```

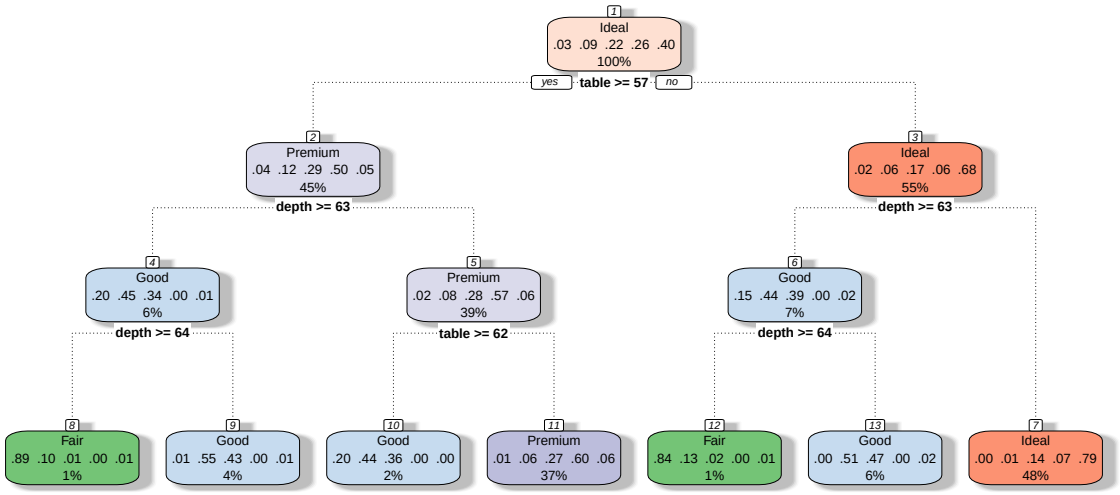
```
## CART
##
## 53940 samples
##     9 predictor
##     5 classes: 'Fair', 'Good', 'Very Good', 'Premium', 'Ideal'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 48546, 48545, 48546, 48546, 48546, 48546, ...
## Resampling results:
##
##   Accuracy   Kappa
##   0.6856322  0.5511799
```

```
pred_tree2 <- predict(tree2, diamonds)
confusionMatrix(diamonds$cut, pred_tree2)
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction   Fair  Good Very Good Premium Ideal
##   Fair       1211   253         0     119    27
##   Good        161  3475         0    1121   149
##   Very Good    16  2960         0    5498  3608
##   Premium       0     0         0   12099  1692
##   Ideal        14    81         0    1184 20272
##
## Overall Statistics
##
##                Accuracy : 0.687
##                  95% CI : (0.6831, 0.6909)
##     No Information Rate : 0.4773
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5543
##
```

```
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Fair Class: Good Class: Very Good Class: Premium
## Sensitivity              0.86377     0.51337               NA         0.6043
## Specificity              0.99241     0.96966            0.776         0.9501
## Pos Pred Value           0.75217     0.70832               NA         0.8773
## Neg Pred Value           0.99635     0.93282               NA         0.8027
## Prevalence               0.02599     0.12549            0.000         0.3712
## Detection Rate           0.02245     0.06442            0.000         0.2243
## Detection Prevalence     0.02985     0.09095            0.224         0.2557
## Balanced Accuracy        0.92809     0.74152               NA         0.7772
##                      Class: Ideal
## Sensitivity                0.7873
## Specificity                0.9546
## Pos Pred Value             0.9407
## Neg Pred Value             0.8309
## Prevalence                 0.4773
## Detection Rate             0.3758
## Detection Prevalence       0.3995
## Balanced Accuracy          0.8710
```

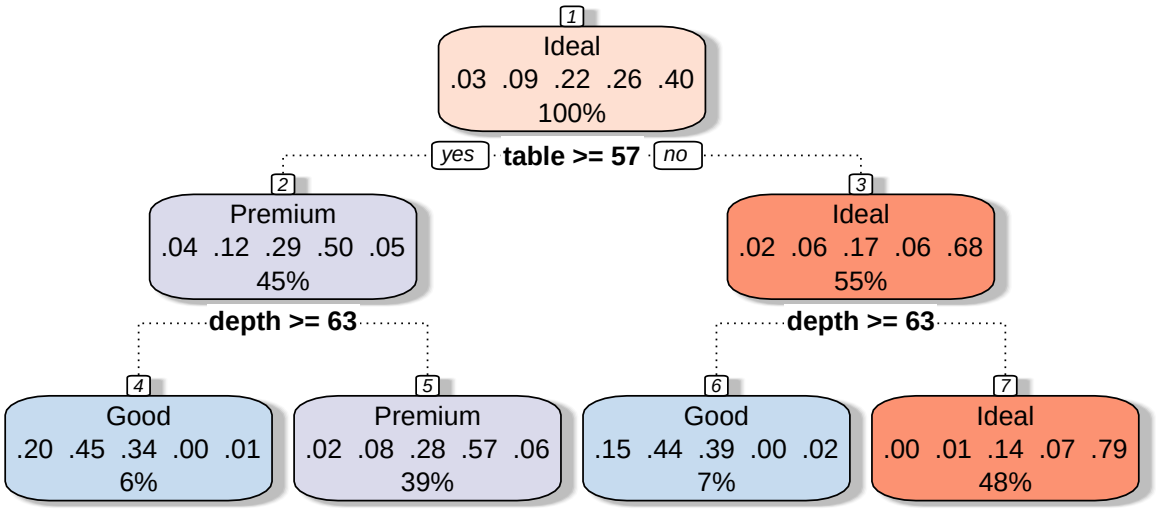
```
fancyRpartPlot(tree2$finalModel, caption = "")
```



```r
#set hyperparameters
hypers = rpart.control(minsplit = 5000, maxdepth = 4, minbucket = 2500)

tree2 <- train(cut ~., data = diamonds, control = hypers, trControl = train_control, method = "rpart1SE
pred_tree2 <- predict(tree2, diamonds)
confusionMatrix(diamonds$cut, pred_tree2)
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction   Fair  Good Very Good Premium Ideal
##   Fair          0  1242         0     341    27
##   Good          0  3157         0    1600   149
##   Very Good     0  2589         0    5885  3608
##   Premium       0     0         0   12099  1692
##   Ideal         0    94         0    1185 20272
##
## Overall Statistics
##
##                Accuracy : 0.6587
##                  95% CI : (0.6546, 0.6627)
##     No Information Rate : 0.4773
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5105
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Fair Class: Good Class: Very Good Class: Premium
## Sensitivity                   NA     0.44578               NA         0.5731
## Specificity              0.97015     0.96267            0.776         0.9485
## Pos Pred Value                NA     0.64350               NA         0.8773
## Neg Pred Value                NA     0.91995               NA         0.7756
## Prevalence               0.00000     0.13129            0.000         0.3914
## Detection Rate           0.00000     0.05853            0.000         0.2243
## Detection Prevalence     0.02985     0.09095            0.224         0.2557
## Balanced Accuracy             NA     0.70423               NA         0.7608
##                      Class: Ideal
## Sensitivity                0.7873
## Specificity                0.9546
## Pos Pred Value             0.9407
## Neg Pred Value             0.8309
## Prevalence                 0.4773
## Detection Rate             0.3758
## Detection Prevalence       0.3995
## Balanced Accuracy          0.8710
```

```r
fancyRpartPlot(tree2$finalModel, caption = "")
```

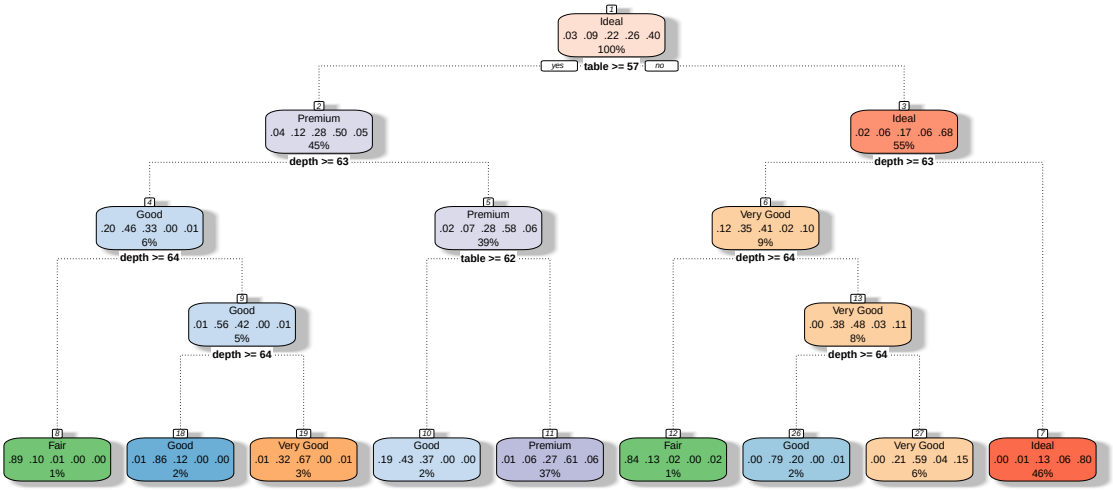



```
#Test vs Train Performances

index = createDataPartition(y = diamonds$cut, p = 0.7, list = FALSE) #partition the data
train_set = diamonds[index,]

test_set = diamonds[-index,]

tree1 <- train(cut ~., data = train_set, method = "rpart1SE", trControl = train_control)
pred_tree <- predict(tree1, test_set)
confusionMatrix(test_set$cut, pred_tree)
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  Fair Good Very Good Premium Ideal
##   Fair       353   82         2      41     5
##   Good        47  735       305     348    36
##   Very Good    3  203       812    1696   910
##   Premium      0    0        42    3623   472
##   Ideal        4    9       147     367  5938
##
## Overall Statistics
##
##                Accuracy : 0.7083
##                  95% CI : (0.7013, 0.7153)
##     No Information Rate : 0.4549
```

```
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5819
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Fair Class: Good Class: Very Good Class: Premium
## Sensitivity              0.86732     0.71429          0.62080         0.5964
## Specificity              0.99176     0.95142          0.81092         0.9491
## Pos Pred Value           0.73085     0.49966          0.22406         0.8758
## Neg Pred Value           0.99656     0.98001          0.96050         0.7964
## Prevalence               0.02515     0.06360          0.08084         0.3755
## Detection Rate           0.02182     0.04543          0.05019         0.2239
## Detection Prevalence     0.02985     0.09091          0.22398         0.2557
## Balanced Accuracy        0.92954     0.83285          0.71586         0.7728
##                      Class: Ideal
## Sensitivity                0.8067
## Specificity                0.9402
## Pos Pred Value             0.9185
## Neg Pred Value             0.8535
## Prevalence                 0.4549
## Detection Rate             0.3670
## Detection Prevalence       0.3996
## Balanced Accuracy          0.8735
```

```
fancyRpartPlot(tree1$finalModel, caption = "")
```

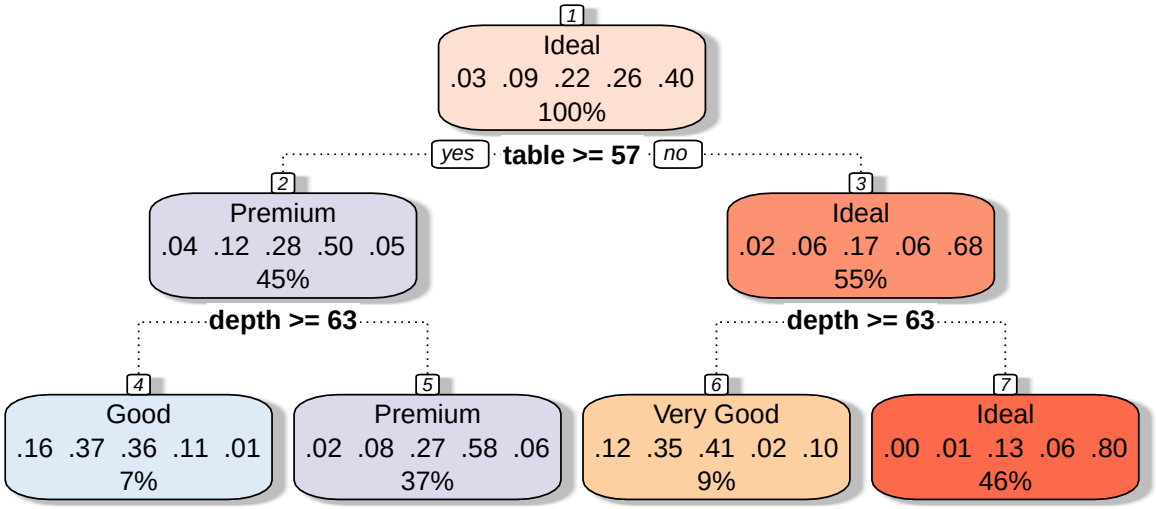



```
hypers = rpart.control(minsplit = 5000, maxdepth = 4, minbucket = 2500)
tree2 <- train(cut ~., data = train_set, control = hypers, trControl = train_control, method = "rpart1SE
pred_tree <- predict(tree2, test_set)
confusionMatrix(test_set$cut, pred_tree)
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  Fair Good Very Good Premium Ideal
##   Fair         0  187       173     118     5
##   Good         0  413       529     493    36
##   Very Good    0  437       593    1684   910
##   Premium      0  136        42    3487   472
##   Ideal        0   16       150     361  5938
##
## Overall Statistics
##
##                Accuracy : 0.6447
##                  95% CI : (0.6373, 0.6521)
##     No Information Rate : 0.4549
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4879
##
##  Mcnemar's Test P-Value : < 2.2e-16
```

```
##
## Statistics by Class:
##
##                      Class: Fair Class: Good Class: Very Good Class: Premium
## Sensitivity                   NA     0.34735          0.39879          0.5676
## Specificity              0.97015     0.92942          0.79371          0.9352
## Pos Pred Value                NA     0.28076          0.16363          0.8429
## Neg Pred Value                NA     0.94724          0.92880          0.7795
## Prevalence               0.00000     0.07349          0.09190          0.3797
## Detection Rate           0.00000     0.02553          0.03665          0.2155
## Detection Prevalence     0.02985     0.09091          0.22398          0.2557
## Balanced Accuracy             NA     0.63839          0.59625          0.7514
##                      Class: Ideal
## Sensitivity                0.8067
## Specificity                0.9402
## Pos Pred Value             0.9185
## Neg Pred Value             0.8535
## Prevalence                 0.4549
## Detection Rate             0.3670
## Detection Prevalence       0.3996
## Balanced Accuracy          0.8735
```

```
fancyRpartPlot(tree2$finalModel, caption = "")
```



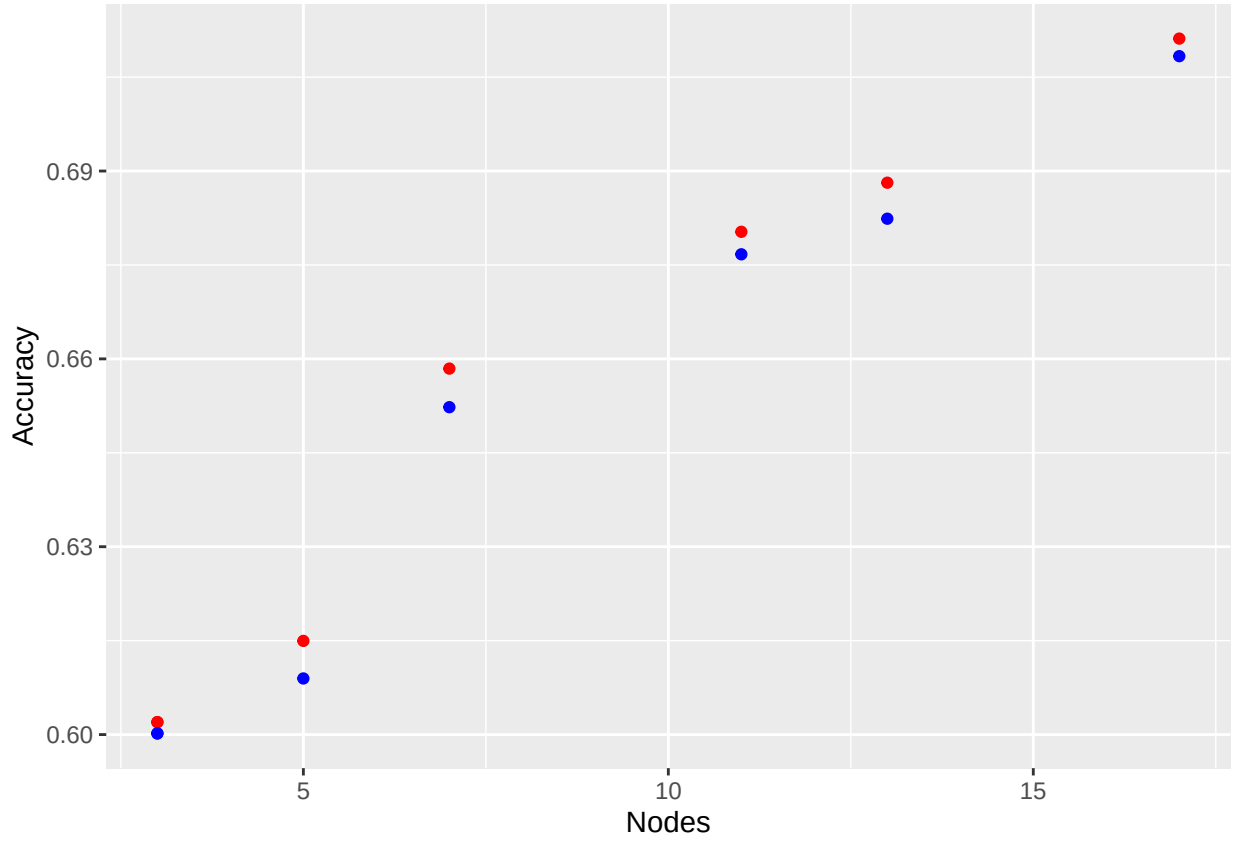
```r
#General model comparison and visualization

train_control = trainControl(method = "cv", number = 10) #initialize cross validation
#tree 1
hypers = rpart.control(minsplit = 2, maxdepth = 1, minbucket = 2)
tree1 <- train(cut ~., data = train_set, control = hypers, trControl = train_control, method = "rpart1SE

#training set
pred_tree <- predict(tree1, train_set)
cfm_train <- confusionMatrix(train_set$cut, pred_tree)

#test set
pred_tree <- predict(tree1, test_set)
cfm_test <- confusionMatrix(test_set$cut, pred_tree)


a_train <- cfm_train$overall[1] #training accuracy
a_test <- cfm_test$overall[1] #testing accuracy
nodes <- nrow(tree1$finalModel$frame) #no of nodes
#from the table
comp_tbl <- data.frame("Nodes" = nodes, "TrainAccuracy" = a_train, "TestAccuracy" = a_test, "MaxDepth" =



#tree 2

hypers = rpart.control(minsplit = 5, maxdepth = 2, minbucket = 5)
tree2 <- train(cut ~., data = train_set, control = hypers, trControl = train_control, method = "rpart1SE

#training set
pred_tree <- predict(tree2, train_set)
cfm_train <- confusionMatrix(train_set$cut, pred_tree)

#test set
pred_tree <- predict(tree2, test_set)
cfm_test <- confusionMatrix(test_set$cut, pred_tree)


a_train <- cfm_train$overall[1] #training accuracy
a_test <- cfm_test$overall[1] #testing accuracy
nodes <- nrow(tree2$finalModel$frame) #no of nodes

comp_tbl <- comp_tbl %>% rbind(list(nodes, a_train, a_test, 2, 5, 5))



#tree 3

hypers = rpart.control(minsplit = 50, maxdepth = 3, minbucket = 50)
tree4 <- train(cut ~., data = train_set, control = hypers, trControl = train_control, method = "rpart1SE

#training set
pred_tree <- predict(tree4, train_set)
```

```r
cfm_train <- confusionMatrix(train_set$cut, pred_tree)

#test set
pred_tree <- predict(tree4, test_set)
cfm_test <- confusionMatrix(test_set$cut, pred_tree)


a_train <- cfm_train$overall[1] #training accuracy
a_test <- cfm_test$overall[1] #testing accuracy
nodes <- nrow(tree4$finalModel$frame) #no of nodes

comp_tbl <- comp_tbl %>% rbind(list(nodes, a_train, a_test, 3, 50, 50))



#tree 4

hypers = rpart.control(minsplit = 100, maxdepth = 4, minbucket = 100)
tree7 <- train(cut ~., data = train_set, control = hypers, trControl = train_control, method = "rpart1SE

#training set
pred_tree <- predict(tree7, train_set)
cfm_train <- confusionMatrix(train_set$cut, pred_tree)

#test set
pred_tree <- predict(tree7, test_set)
cfm_test <- confusionMatrix(test_set$cut, pred_tree)


a_train <- cfm_train$overall[1] #training accuracy
a_test <- cfm_test$overall[1] #testing accuracy
nodes <- nrow(tree7$finalModel$frame) #no of nodes

comp_tbl <- comp_tbl %>% rbind(list(nodes, a_train, a_test, 4, 100, 100))



#tree 5


hypers = rpart.control(minsplit = 1000, maxdepth = 4, minbucket = 1000)
tree8 <- train(cut ~., data = train_set, control = hypers, trControl = train_control, method = "rpart1SE

#training set
pred_tree <- predict(tree8, train_set)
cfm_train <- confusionMatrix(train_set$cut, pred_tree)

#test set
pred_tree <- predict(tree8, test_set)
cfm_test <- confusionMatrix(test_set$cut, pred_tree)


a_train <- cfm_train$overall[1] #training accuracy
```

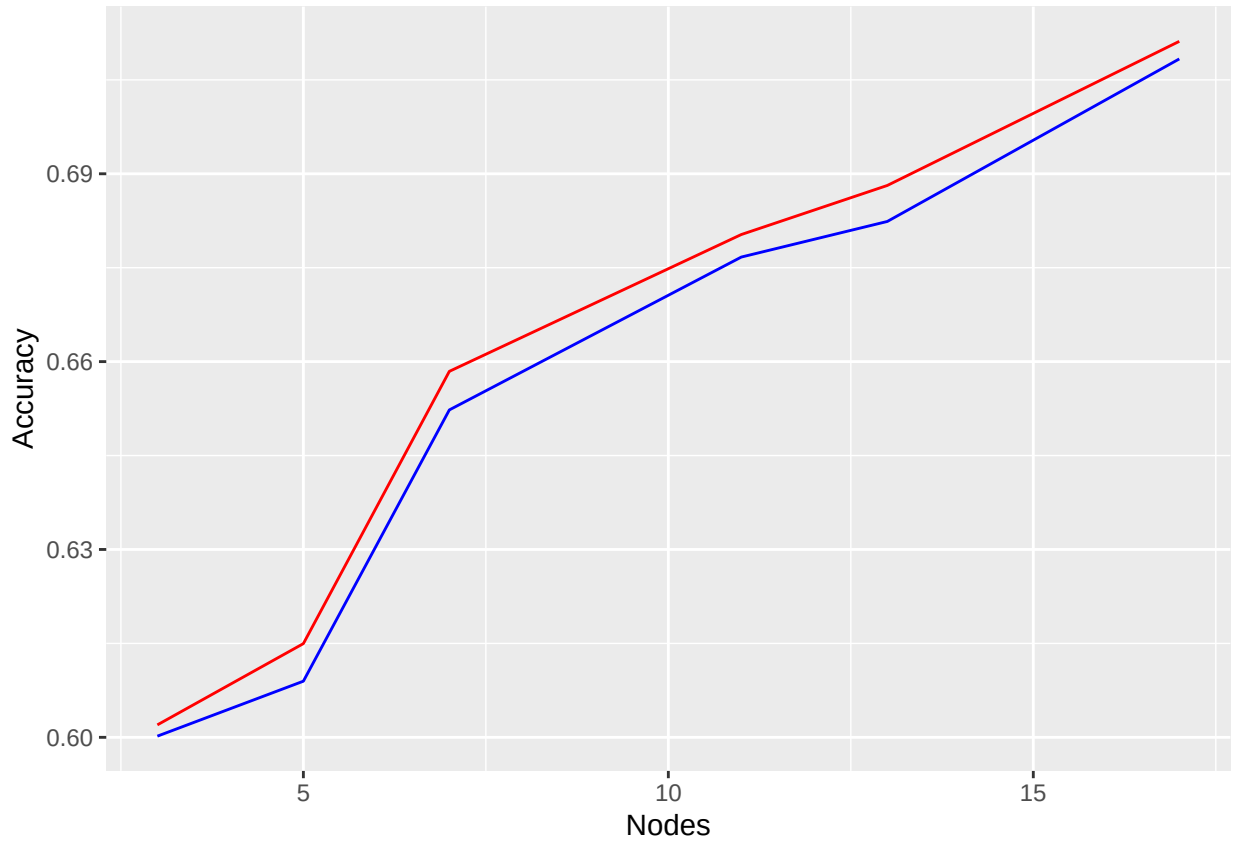
```r
a_test <- cfm_test$overall[1] #testing accuracy
nodes <- nrow(tree8$finalModel$frame) #no of nodes

comp_tbl <- comp_tbl %>% rbind(list(nodes, a_train, a_test, 4, 1000, 1000))



#tree 6


hypers = rpart.control(minsplit = 5000, maxdepth = 8, minbucket = 5000)
tree10 <- train(cut ~., data = train_set, control = hypers, trControl = train_control, method = "rpart1

#training set
pred_tree <- predict(tree10, train_set)
cfm_train <- confusionMatrix(train_set$cut, pred_tree)

#test set
pred_tree <- predict(tree10, test_set)
cfm_test <- confusionMatrix(test_set$cut, pred_tree)


a_train <- cfm_train$overall[1] #training accuracy
a_test <- cfm_test$overall[1] #testing accuracy
nodes <- nrow(tree10$finalModel$frame) #no of nodes

comp_tbl <- comp_tbl %>% rbind(list(nodes, a_train, a_test, 8, 5000, 5000))



#tree 7


hypers = rpart.control(minsplit = 10000, maxdepth = 25, minbucket = 10000)
tree11 <- train(cut ~., data = train_set, control = hypers, trControl = train_control, method = "rpart1

#training set
pred_tree <- predict(tree11, train_set)
cfm_train <- confusionMatrix(train_set$cut, pred_tree)

#test set
pred_tree <- predict(tree11, test_set)
cfm_test <- confusionMatrix(test_set$cut, pred_tree)


a_train <- cfm_train$overall[1] #training accuracy
a_test <- cfm_test$overall[1] #testing accuracy
nodes <- nrow(tree11$finalModel$frame) #no of nodes

comp_tbl <- comp_tbl %>% rbind(list(nodes, a_train, a_test, 25, 10000, 10000))

comp_tbl
```

```
##          Nodes TrainAccuracy TestAccuracy MaxDepth Minsplit Minbucket
## Accuracy     3     0.6019862    0.6001854        1        2         2
## 1            7     0.6584481    0.6522868        2        5         5
## 11          13     0.6881356    0.6823857        3       50        50
## 12          17     0.7111494    0.7083436        4      100       100
## 13          11     0.6802966    0.6766996        4     1000      1000
## 14           5     0.6149629    0.6089617        8     5000      5000
## 15           3     0.6019862    0.6001854       25    10000     10000
```

```
#with scatter plot

ggplot(comp_tbl, aes(x = Nodes)) +
  geom_point(aes(y = TrainAccuracy), color = "red") +
  geom_point(aes(y = TestAccuracy), color = "blue") +
  ylab("Accuracy")
```




```
#with line plot

ggplot(comp_tbl, aes(x = Nodes)) +
  geom_line(aes(y = TrainAccuracy), color = "red") +
  geom_line(aes(y = TestAccuracy), color = "blue") +
  ylab("Accuracy")
```




```
# FEATURE SELECTION

colnames(diamonds)
```

```
##  [1] "carat"   "cut"     "color"   "clarity" "depth"   "table"   "price"
##  [8] "x"       "y"       "z"
```

```
tree1 <- train(cut ~., data = train_set, method = "rpart1SE", trControl = train_control)

var_imp <- varImp(tree1, scale = FALSE) #view variable imp sources
var_imp
```

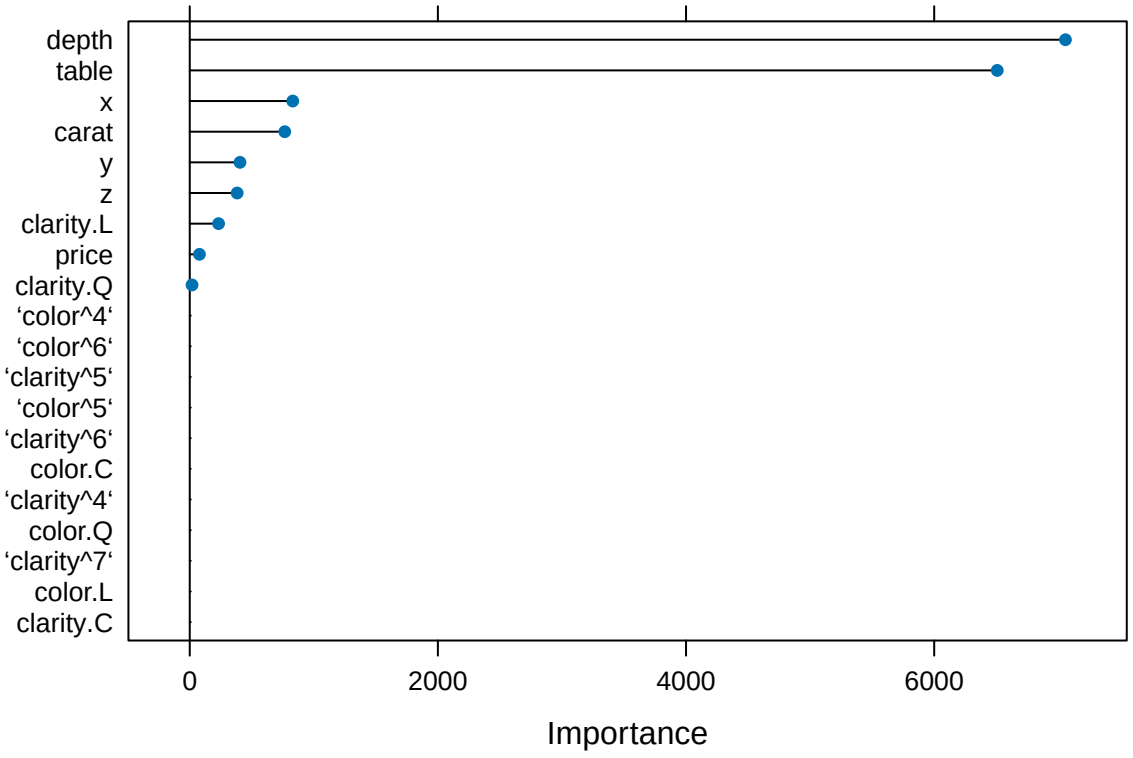
```
## rpart1SE variable importance
##
##             Overall
## depth       7057.99
## table       6508.14
## x            830.03
## carat        765.81
## y            404.91
## z            382.21
## clarity.L    232.47
## price         78.28
## clarity.Q     18.46
## color.L        0.00
```

```
## color.Q        0.00
## `clarity^6`    0.00
## color.C        0.00
## `clarity^7`    0.00
## `color^5`      0.00
## `clarity^5`    0.00
## clarity.C      0.00
## `color^4`      0.00
## `clarity^4`    0.00
## `color^6`      0.00
```

```
#estimate variable importance
importance <- varImp(tree1, scale = FALSE)
print(importance) #summarize imp
```

```
## rpart1SE variable importance
##
##             Overall
## depth       7057.99
## table       6508.14
## x            830.03
## carat        765.81
## y            404.91
## z            382.21
## clarity.L    232.47
## price         78.28
## clarity.Q     18.46
## `clarity^4`    0.00
## clarity.C      0.00
## `clarity^5`    0.00
## `color^6`      0.00
## `color^4`      0.00
## color.Q        0.00
## color.L        0.00
## color.C        0.00
## `clarity^7`    0.00
## `clarity^6`    0.00
## `color^5`      0.00
```

```
#visualize
plot(importance)
```




```
library(mlbench)
data("PimaIndiansDiabetes")

index = createDataPartition(y = PimaIndiansDiabetes$diabetes, p = 0.7, list = FALSE) #partition the dat

train_set = PimaIndiansDiabetes[index,]
test_set = PimaIndiansDiabetes[-index,]

library(randomForest)
```

```
## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:rattle':
##
##     importance

## The following object is masked from 'package:ggplot2':
##
##     margin
```

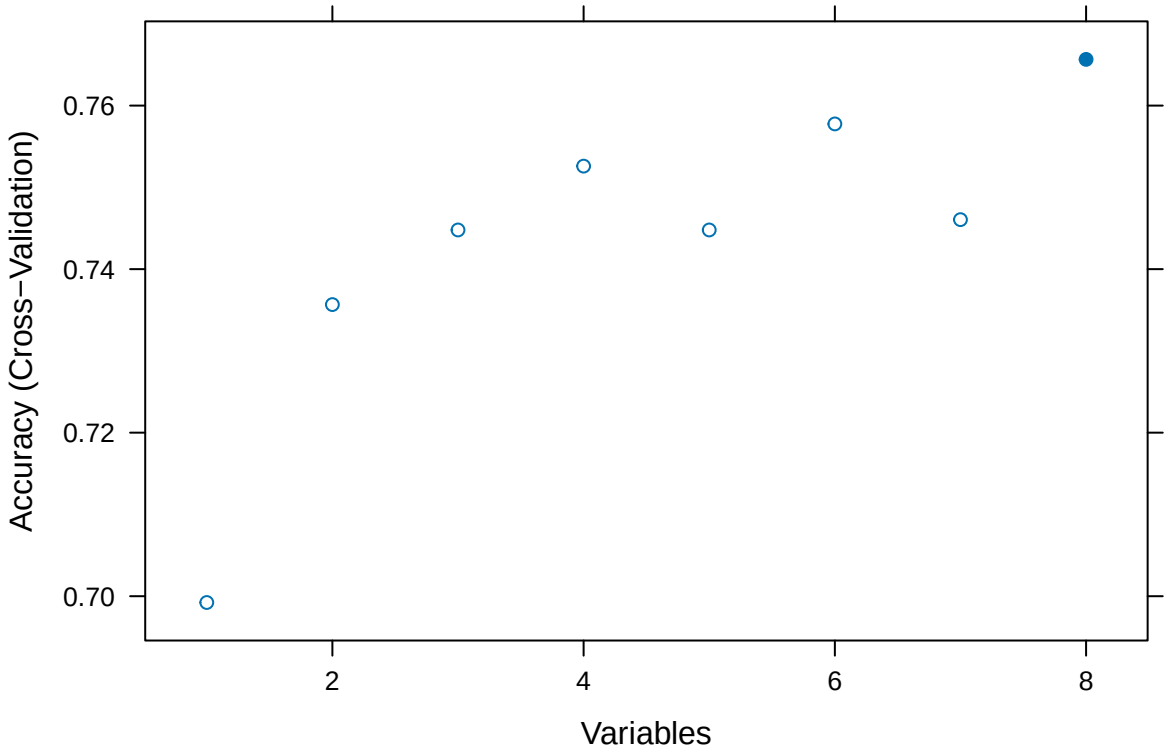
```r
control <- rfeControl(functions = rfFuncs, method = "cv", number = 10) #define the control using random

results <- rfe(x = PimaIndiansDiabetes[,1:8], y = PimaIndiansDiabetes[,9], sizes = c(1:8), rfeControl =

print(results) #summarize
```

```
##
## Recursive feature selection
##
## Outer resampling method: Cross-Validated (10 fold)
##
## Resampling performance over subset size:
##
##  Variables Accuracy  Kappa AccuracySD KappaSD Selected
##          1   0.6992 0.2850    0.03534 0.07110
##          2   0.7357 0.3950    0.04764 0.09887
##          3   0.7448 0.4270    0.05990 0.12551
##          4   0.7526 0.4443    0.06707 0.14302
##          5   0.7448 0.4276    0.05927 0.13038
##          6   0.7578 0.4528    0.05806 0.12380
##          7   0.7461 0.4262    0.05128 0.11092
##          8   0.7657 0.4666    0.04853 0.10962        *
##
## The top 5 variables (out of 8):
##    glucose, mass, age, pregnant, insulin
```

```r
predictors(results) #list the chosen features in order
```

```
## [1] "glucose"  "mass"     "age"      "pregnant" "insulin"  "pedigree" "triceps"
## [8] "pressure"
```

```r
plot(results) #plot the results
```


Accuracy (Cross−Validation)
0.76
0.74
0.72
0.70
2
4
6
8
Variables